“Smart CPU Scheduler with Dynamic Priority Adjustment”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

VARSHITH COMPLE	**2520080016**
RISHITH	**2520080049**
MOHITH KARTHIK	**2520080059**

Under the Esteemed Guidance of

Dr. M.Raghupathi
Assistant Professor
Department of Artificial Intelligence and Data Science

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Smart CPU Scheduler with Dynamic Priority Adjustment"** is a record of Bonafide work of **Varshith Comple-2520080016, Rishith-2520080049, Mohith Karthik-2520080059** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

VARSHITH COMPLE	**2520080016**
RISHITH	**2520080049**
MOHITH KARTHIK	**2520080059**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled**"Smart CPU Scheduler with Dynamic Priority Adjustment"** is a bonafide work done and submitted by **Varshith Comple-2520080016, Rishith-2520080049, Mohith Karthik-2520080059,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Artificial Intelligence and Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr. M. Raghupathi**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

VARSHITH COMPLE	**2520080016**
RISHITH	**2520080049**
MOHITH KARTHIK	**2520080059**

Abstract

The CPU is shared by many processes, and the scheduler decides which process runs next. Classic priority scheduling assigns each process a fixed priority. Under this approach, low-priority processes can wait for a very long time while higher-priority processes keep arriving, a problem known as starvation. Fixed priorities also make interactive applications feel slow when they must queue behind CPU-heavy work.

This project implements a Smart CPU Scheduler whose priorities change while the system runs. Each task has a dynamic score made of its base priority, an aging bonus that grows while it waits, an I/O boost after it waits for input, and a CPU penalty after heavy use. The scheduler always runs the task with the highest score.

The project is delivered as a full-stack web application. A dark, responsive landing page explains the idea and lists the operating systems course outcomes. Users can sign up and log in. A protected dashboard lets them define tasks, run a side-by-side simulation of base-priority scheduling and dynamic-priority scheduling, view Gantt-style timelines, and save or delete their runs. The backend is built with Node.js and Express, uses bcrypt for password hashing and JSON Web Tokens for sessions, and stores data in an SQLite database.

The simulation shows that aging prevents a low-priority task from being starved, and that the trade-off is a small delay for other tasks. The project is a teaching and demonstration tool for process scheduling. It does not modify a kernel, and it does not measure real processes.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		9-10
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background		10-12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		14-15
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	

5	**System Architecture and Design**		**16-19**
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**20-22**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**23-26**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**27-28**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**29**
		9.1 Books	
		9.2 Research Papers	

		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

An operating system lets many programs share one processor. Each program, or process, takes turns on the CPU. The part of the operating system that decides which process runs next, and for how long, is the CPU scheduler. Scheduling affects how fast the system responds, how fairly work is shared, and how efficiently the processor is used.

Many scheduling methods are taught in operating systems courses, including first-come first-served, shortest job first, round robin, and priority scheduling. Priority scheduling is widely used because it lets important work run first. However, it is sensitive to how priorities are assigned. This project studies that sensitivity and proposes a way to correct it.

1.2 Problem Statement

In a basic priority scheduler, each process keeps a fixed priority. The scheduler always chooses the ready process with the highest priority. This causes two main problems:

- Starvation: a low-priority process may never receive the CPU if higher-priority processes keep arriving.
- Poor response time: an interactive process, such as a video call or an editor, may wait behind CPU-heavy work even though it needs only short bursts of CPU time.

A further difficulty is that students and developers have few simple tools for seeing how such behavior changes under different policies.

1.3 Scope of the Project

The project covers the following:

- A simulation of a single CPU that runs tasks with base priorities, I/O waits, and CPU-heavy work.
- A dynamic priority rule with aging, I/O boost, and CPU penalty.
- A side-by-side comparison of base-priority scheduling and dynamic-priority scheduling.
- A web application with a landing page, user sign up, log in, a protected dashboard, and saved simulation runs.
- A backend API and an SQLite database that store users and saved runs.

The project does not modify a real operating system kernel, does not model multi-core processors, and does not measure real processes running on the computer. A C implementation that uses real threads and system calls is listed as future work.

1.4 Objectives of the Project

1. To study the concepts of process scheduling, starvation, and synchronization.
2. To design a scheduling policy whose priorities adjust while the system runs.
3. To implement a simulator that runs both the static and dynamic policies on the same workload.
4. To present the results clearly, using waiting-time tables and timeline charts.
5. To build a secure web application where users can run and save simulations.

1.5 Importance of the Application

The application helps learners see scheduling behavior rather than only read about it. A student can change a task's priority or type, run the simulation, and immediately see the effect on waiting time and on the order in which tasks run. Instructors can use the same tool for demonstrations. The saved-runs feature lets students keep their experiments for later comparison.

1.6 Target Users / Audience

- Undergraduate students taking an operating systems or systems programming course.
- Instructors who want a visual demonstration of scheduling policies.
- Developers who want to understand the trade-offs of priority-based scheduling.

Chapter -2 OS Concepts and Background

2.1 Operating System Concepts Related to the Project

The project relies on the following operating system concepts, which are covered in the course outcomes CO-2 and CO-6.

- Process states: a process is ready (waiting for the CPU), running (using the CPU), blocked (waiting for input or output), or terminated.
- CPU scheduling: the policy that chooses the next ready process and the length of its time slice.
- Priority: a number that expresses how important a process is. Higher values usually run first.
- Starvation: a situation in which a ready process never receives the CPU.
- Aging: a technique that gradually increases the priority of a process that keeps waiting, so that it eventually runs.
- Synchronization: the coordination of processes or threads that share data. Mutexes protect a shared data structure such as a ready queue when several threads access it.

2.2 Existing Methods / Algorithms

Table 2.1 compares the common scheduling methods discussed in operating systems literature.

Method	Main idea	Advantage	Limitation
First-come first-served	Runs processes in arrival order	Simple to implement	Long jobs delay short ones
Shortest job first	Runs the shortest expected job next	Low average waiting time	Needs job length in advance; long jobs may starve
Round robin	Gives each ready process a fixed time slice	Fair and responsive	Frequent switches add overhead
Static priority	Runs the highest fixed-priority process	Important work runs first	Starvation of low-priority tasks
Multilevel feedback queue	Moves processes between queues based on behavior	Adapts to behavior	Many parameters to tune

Table 2.1: Common CPU scheduling methods.

Static priority scheduling is the baseline for this project because it is simple and widely taught. Multilevel feedback queues are the closest existing method in spirit, since they adjust a process's level based on its behavior. The proposed approach is a simpler adjustment that uses a single score per task.

2.3 Proposed Approach

Each task receives a dynamic score at every scheduling decision:

dynamic score = base priority + aging + I/O boost - CPU penalty
The three adjustment rules work as follows:

- Aging: every tick that a ready task waits without running, its aging value increases by one, up to a limit of 20. When the task runs, its aging value is reset to zero.
- I/O boost: an I/O task pauses for two ticks after running for two ticks, as if it were waiting for input. When it returns, it receives a boost of 3, which fades by one each tick.
- CPU penalty: each time a task runs, its penalty increases by two, up to a limit of 10. The penalty fades by one each tick while the task is not running.

With these rules, a task that waits long enough eventually has the highest score and runs. A task that uses the CPU continuously gradually loses priority to others. An interactive task that is waiting for input regains priority quickly when it returns.

2.4 Advantages of the Proposed Approach

- No task is starved, because a waiting task’s score keeps rising.
- Interactive tasks regain priority after waiting for input, which shortens their wait.
- CPU-heavy tasks share the processor with others instead of monopolizing it.
- The method needs only three integer values per task and a few additions, so it is easy to understand and explain.
- The same workload can be run with and without the adjustment, which makes the effect easy to measure.

Chapter -3 System Requirements

3.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Any 64-bit processor	Apple Silicon or Intel i5 or better
Memory (RAM)	4 GB	8 GB
Free disk space	300 MB (includes dependencies)	1 GB
Display	1280 x 720	1920 x 1080

Table 3.1: Hardware requirements.

3.2 Software Requirements

- Node.js version 22.13 or newer. The project uses the SQLite module built into Node.js.
- npm, which is installed with Node.js.
- A terminal application (Terminal on macOS, or any shell on Linux and Windows Subsystem for Linux).
- A current web browser such as Chrome, Safari, Edge, or Firefox.

3.3 Operating System Environment

The application was set up and run on macOS on Apple Silicon. The backend API was also tested in a Linux environment, using the same commands. Because the project runs on Node.js, it works on macOS, Linux, Ubuntu on Windows Subsystem for Linux, and Windows. No operating system specific code is used.

3.4 Development Tools and Libraries

Tool / library	Version used	Purpose
Node.js	22.22 (tested); 26.x (user machine)	Runs the JavaScript backend
Express	4.19	Web server and API routing
bcryptjs	2.4	Password hashing
jsonwebtoken	9.0	Login tokens
node:sqlite	Built into Node.js 22.13+	SQLite database access
Visual Studio Code	Optional	Editing the source files

Chapter - 4 Technology and Tools Used

4.1 JavaScript

JavaScript is used for both the browser and the server. The browser code runs the landing page, the forms, and the scheduler simulator. The server code runs under Node.js and handles requests, authentication, and database access. Using one language across the project reduces the number of tools a student must learn.

4.2 Compiler

No compiler is used in this version. JavaScript is interpreted by Node.js and by the browser engine, so the project has no build step. The course outline mentions GCC and C/C++. These are part of the planned future work, described in Chapter 8, and are not used in the current application.

4.3 System Calls and Node.js APIs

The application does not call Linux system calls directly. The Node.js runtime performs file and network operations on its behalf through a portability layer, and the application uses the following Node.js APIs:

- fs and path: to read the schema file and to create the database folder.
- http (through Express): to accept requests from browsers.
- node:sqlite: to open the database file and run SQL statements.

4.4 Concurrency and Synchronization

Node.js runs JavaScript on a single thread with an event loop, so the application does not create POSIX threads and does not need mutexes to protect its data. The scheduler simulation models concurrency logically: the list of ready tasks plays the role of a shared ready queue. Real thread synchronization is part of the future work.

4.5 Commands and Utilities

Command	Purpose
cd	Moves into the backend folder
npm install	Downloads the libraries listed in package.json
npm start	Starts the server
curl	Tests API endpoints from the terminal

Table 4.1: Commands used.

4.6 Other Tools and Libraries

- Manrope typeface, loaded from Google Fonts, for the page text.
- A CSS grid layout for the Gantt-style timeline, with no charting library.
- Browser localStorage to keep the login token between page visits.

Chapter - 5 System Architecture and Design

5.1 Overall System Architecture

The system follows a three-layer design. The presentation layer runs in the browser and consists of HTML, CSS, and JavaScript files. The application layer is an Express server that exposes a REST-style API and checks login tokens. The data layer is an SQLite database file that stores users and saved runs. The scheduler simulation runs in the browser, so the server only stores and returns results.

Layer	Technology	Responsibility
Presentation	HTML, CSS, JavaScript	Pages, forms, simulator, timelines
Application	Node.js, Express, bcryptjs, jsonwebtoken	Authentication, saved runs, syllabus data
Data	SQLite (node:sqlite)	Users and scheduler runs

Table 5.1: System layers.

5.2 System Workflow

1. A visitor opens the landing page, which loads the course outcomes from the API.
2. The visitor creates an account. The server hashes the password, stores the user, and returns a token.
3. The browser stores the token and opens the dashboard.
4. The dashboard asks the server who is logged in. If the token is missing or expired, the user is sent to the login page.
5. The user enters tasks and clicks Run simulation. The browser runs both the base-priority and dynamic-priority simulations.
6. The user clicks Save run. The browser sends the tasks and results to the server, which stores them under the user’s account.
7. The user can load or delete saved runs at any time.
8. Logging out removes the token from the browser.

5.3 Flowchart / Data Flow

The data flow for a single simulation run is shown in Table 5.2. Each row is one step, read from top to bottom.

Step	From	To	Data
1	User	Dashboard form	Task names, types, base priorities, work ticks
2	Dashboard form	Scheduler (browser)	Validated task list
3	Scheduler (browser)	Dashboard results	Base-priority and dynamic timelines and summaries
4	Dashboard results	Express API	Run title, configuration, results (on save)
5	Express API	SQLite database	Row in scheduler_runs linked to the user
6	SQLite database	Express API	Saved runs for the user
7	Express API	Dashboard	Saved-runs list shown on the page

5.4 Module Description

Table 5.3: Module description.

File	Module	Responsibility
backend/server.js	Server	Starts Express, mounts routes, serves the frontend, handles errors
backend/db.js	Database	Opens the SQLite file and creates tables from schema.sql
backend/middleware/auth.js	Authentication	Signs tokens and blocks requests without a valid token
backend/routes/auth.js	Authentication API	Sign up, log in, and current user
backend/routes/runs.js	Runs API	List, save, and delete the user's runs
backend/routes/syllabus.js	Syllabus API	Returns the course outcomes
frontend/js/api.js	Client API	Sends requests and manages the stored session
frontend/js/main.js	Shared UI	Element builder, navigation state, logout
frontend/js/auth.js	Forms	Sign up and log in form handling
frontend/js/scheduler.js	Simulator	Runs the base-priority and dynamic-priority simulations
frontend/js/dashboard.js	Dashboard	Task editor, results, Gantt charts, saved runs
database/schema.sql	Schema	Table and index definitions

5.5 Input and Output Design

Table 5.4 lists the inputs accepted by the system and the rules applied to them.

Input	Where entered	Rules
Name (sign up)	Sign-up form	At least 2 characters
Email	Sign-up and log-in forms	Must look like an email address; stored in lowercase
Password	Sign-up and log-in forms	At least 8 characters at sign up
Task name	Dashboard	Required, up to 20 characters, unique within a run
Task type	Dashboard	CPU or I/O
Base priority	Dashboard	Whole number from 0 to 20
Work (ticks)	Dashboard	Whole number from 1 to 30
Run name	Dashboard	Up to 80 characters; a default name is used if empty

Table 5.4: Input rules.

The outputs are the response messages shown to the user and the results of each simulation. Table 5.5 describes the outputs.

Output	Shown where	Content
Error message	Forms and dashboard	The reason the input was rejected, such as an invalid email
Summary table	Dashboard results	Per task: base, waiting before, waiting after, finish before, finish after
Gantt timelines	Dashboard results	One row per task and one column per tick, filled when the task runs
Saved-runs list	Dashboard	Run name, date, total waiting before and after, Load and Delete buttons
Course outcomes	Landing page	Six outcomes with topics; project topics highlighted

Database design

The database holds two tables. The users table stores one row per account. The scheduler_runs table stores one row per saved run, with the tasks and the results as JSON text. Each run is linked to a user, and deleting a user also deletes that user’s runs.

Table	Column	Type	Notes
users	id	INTEGER	Primary key, auto-increment
users	name, email	TEXT	Email is unique
users	password_hash	TEXT	bcrypt hash, never the plain password
users	created_at	TEXT	Set automatically
scheduler_runs	id	INTEGER	Primary key, auto-increment
scheduler_runs	user_id	INTEGER	Foreign key to users.id
scheduler_runs	title, config, result	TEXT	config and result are JSON
scheduler_runs	created_at	TEXT	Set automatically

Chapter - 6 Implementation

6.1 Module-wise Implementation

The implementation is divided into seven modules: the server, the database layer, authentication, saved runs, the syllabus, the simulator, and the dashboard. Each is described below.

- Server: loads the middleware and routes, serves static files from the frontend folder, and returns JSON errors. The request body size is limited to 500 KB.
- Database layer: opens app.db, enables foreign keys and write-ahead logging, and runs schema.sql so the tables exist on first start.
- Authentication: signup validates the name, email, and password, rejects duplicate emails, hashes the password with bcrypt, and returns a signed token valid for seven days. Login compares the password with the stored hash and gives the same error for a wrong email and a wrong password, so attackers cannot learn which accounts exist.
- Saved runs: every route requires a valid token. Each query includes the user's id, so a user cannot read or delete another user's run.
- Syllabus: returns the course outcomes as static data.
- Simulator: a pure function that takes the task list and an option for dynamic or base-only scheduling and returns a timeline and a summary.
- Dashboard: builds the task editor, validates inputs, calls the simulator twice, renders the results, and manages saved runs.

6.2 Algorithm / Pseudocode

The simulation advances one tick at a time. The procedure below is the same as the implemented function simulate in frontend/js/scheduler.js.

```
procedure SIMULATE(tasks, dynamic):
    for each task: remaining <- burst; aging, penalty, boost, blocked, waited <- 0
    for tick <- 1 to MAX_TICKS:
        if every task has remaining = 0: stop
        for each task blocked for input:
            decrease its blocked counter
            if counter reached 0 and dynamic: boost <- 3
        runner <- ready task (remaining > 0, not blocked) with highest score
                  ties go to the task with the smaller name
        for each ready task other than runner:
            waited <- waited + 1
            if dynamic: aging <- min(aging + 1, 20)
        if runner exists:
            remaining <- remaining - 1
            if dynamic: aging <- 0; boost <- 0; penalty <- min(penalty + 2, 10)
            if remaining = 0: record finish tick
```

```
            else if runner is I/O and it ran 2 ticks in a row: block it for 2 ticks
        if dynamic: for each task other than runner: penalty and boost decrease by
1 (not below 0)
        record (tick, runner)
    return timeline and per-task summary

score(task) = base + aging + boost - penalty   if dynamic
score(task) = base                              if not dynamic
```

The base-only run uses the same loop with the score fixed at the base priority. This makes the two runs directly comparable: the only difference is the adjustment rules.

6.3 Core Implementation

The score and the runner selection are the core of the scheduler. The actual code is shown below.

```
const score = p => (dynamic ? p.base + p.aging + p.boost - p.penalty : p.base);

let runner = null;
procs.forEach(p => {
  if (p.remaining === 0 || p.blockedFor > 0) return;
  if (!runner || score(p) > score(runner) ||
      (score(p) === score(runner) && p.name < runner.name)) {
    runner = p;
  }
});
```

Aging is applied to every ready task that does not run in the current tick:

```
procs.forEach(p => {
  if (p === runner || p.remaining === 0 || p.blockedFor > 0) return;
  p.waitTicks++;
  if (dynamic) p.aging = Math.min(p.aging + 1, 20);
});
```

6.4 APIs Used

API	Where used	Purpose
Express routing	backend/routes	Defines the API endpoints
express.json middleware	backend/server.js	Reads JSON request bodies
express.static	backend/server.js	Serves the frontend files
node:sqlite DatabaseSync	backend/db.js	Opens the database and runs SQL
jsonwebtoken sign and verify	backend/middleware/auth.js	Creates and checks login tokens
bcryptjs hashSync and compareSync	backend/routes/auth.js	Hashes and checks passwords
Fetch API	frontend/js/api.js	Sends requests from the browser
localStorage	frontend/js/api.js	Keeps the token between visits
DOM API	frontend/js/main.js	Builds the page elements safely

6.5 Process / Thread Implementation

The project does not create operating system processes or threads. Each task in the simulation is a plain JavaScript object with a name, a type, a base priority, a remaining burst time, and several counters. These objects represent processes in the model. The scheduler decides which object to run in each tick. The behavior is comparable to a real scheduler, but no real process is started, paused, or resumed.

6.6 IPC / Synchronization Implementation

Inter-process communication is not used in this version. The simulation runs entirely inside one browser tab, and the server communicates with the browser only through HTTP requests. Synchronization is not needed because JavaScript runs one task at a time on a single thread. The shared ready list is updated only by the scheduler loop. A mutex-based implementation with real threads is listed as future work.

6.7 Memory Management Implementation

Memory is managed by the JavaScript engine and the Node.js runtime, so the project does not allocate or free memory manually. The project does, however, control how much data it keeps:

- Passwords are stored only as bcrypt hashes.
- The request body is limited to 500 KB, so one request cannot consume excessive memory.
- The simulation is limited to 8 tasks, 30 ticks of work per task, and 100 ticks in total, which keeps every run small.
- Each run’s timeline is stored as JSON in the database, so memory use per run is bounded.

6.8 User Interface Implementation

The interface uses a dark theme, with a peach accent color for the main buttons. The landing page has a hero section with a cluster of glossy spheres drawn with CSS. The dashboard uses a two-column layout: the task editor on the left and the results on the right. The Gantt-style timelines are built with a CSS grid, where each filled cell marks a tick in which that task ran. The page also respects the reduced-motion setting and collapses to a single column on small screens.

Chapter - 7 Execution and Demonstration

7.1 Setup and Compilation

No compilation is required. The setup steps on macOS are as follows, and they are the same on Linux and Windows Subsystem for Linux:

1. Install Node.js version 22.13 or newer from nodejs.org, or with Homebrew (brew install node).
2. Unzip the project folder, cpu-scheduler-website, into the Downloads folder.
3. Open Terminal and move into the backend folder: cd ~/Downloads/cpu-scheduler-website/backend.
4. Install the dependencies with npm install.

7.2 Execution Commands

```
cd ~/Downloads/cpu-scheduler-website/backend
npm install
npm start

Output:
Smart CPU Scheduler is running at http://localhost:3000
```

Open http://localhost:3000 in a browser. Create an account, open the dashboard, and run the simulation. Press Ctrl + C in Terminal to stop the server.

The API can also be tested from another terminal window. For example, the following command creates an account:

7.3 Sample Inputs and Outputs

The dashboard starts with three default tasks. The table below lists them, followed by the results of running them through the simulator. The numbers are produced by the program and are not estimates.

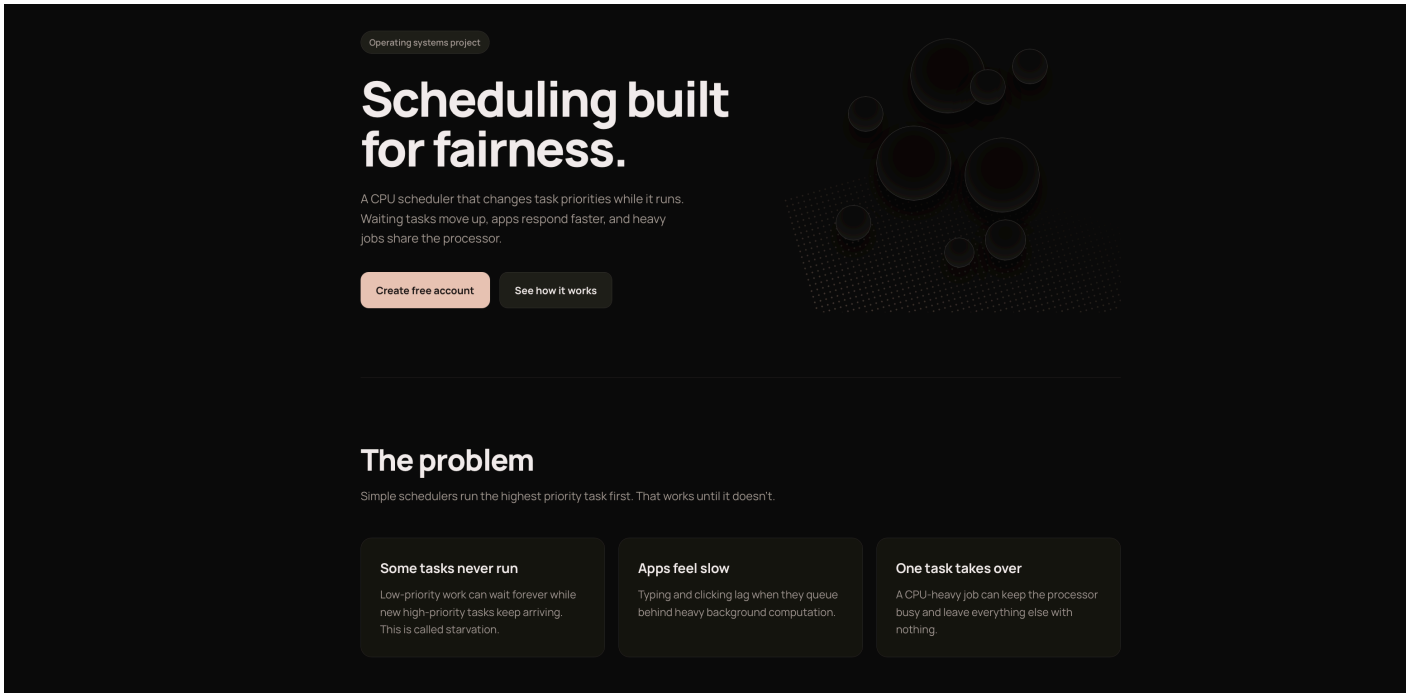
Task	Type	Base priority	Work (ticks)
Video call	I/O	5	6
Math job	CPU	3	10
Backup	CPU	1	6

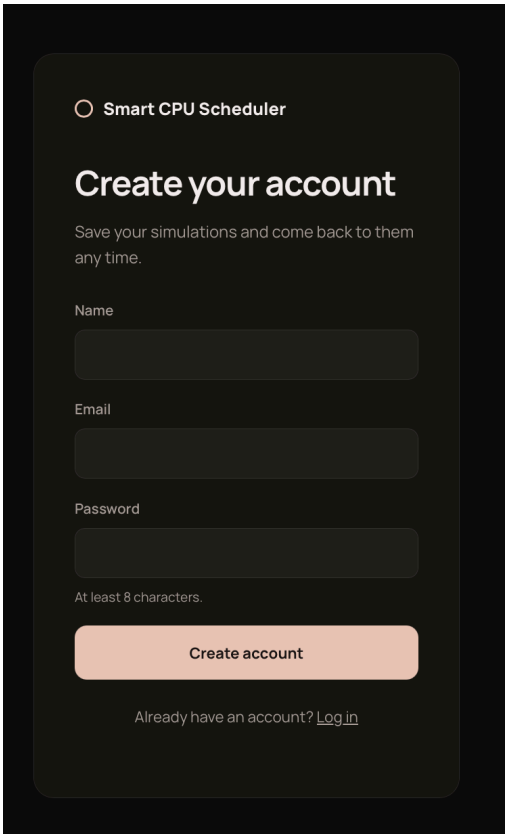
Timeline strings below use V for Video call, M for Math job, B for Backup, and each character is one tick. A dash would mean the CPU was idle, but no idle ticks occurred in these runs.

Task	Waiting (base only)	Finished (base only)	Waiting (dynamic)	Finished (dynamic)
Video call	0	Tick 8	5	Tick 13
Math job	6	Tick 16	12	Tick 22
Backup	16	Tick 22	12	Tick 18

The results show the intended effect on the starved task. Under base-priority scheduling, Backup waits 16 ticks and finishes last, at tick 22. Under dynamic priority, its waiting time drops to 12 and it finishes at tick 18. The results also show the trade-off. The Video call task, which is an interactive task, waits 5 ticks in the dynamic run, compared with 0 ticks under base priority, and finishes later. The dynamic policy therefore protects the starved task at some cost to the interactive one. Changing the I/O boost or the aging limit changes this balance, and the dashboard lets a user compare the two runs for any task list.

7.4 Screenshots of Execution





Scheduler simulator

Add tasks, run the simulation, and compare base priority with dynamic priority.

Tasks

CPU tasks use the processor continuously. I/O tasks pause to wait for input.

Name	Type	Base priority	Work (ticks)	
Video call	I/O	5	6	×
Math job	CPU	3	10	×
Backup	CPU	1	6	×

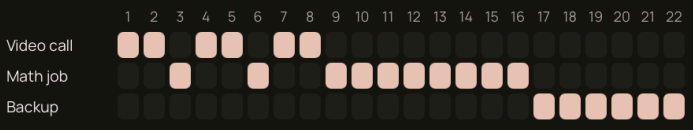
Add task | Run simulation

Results

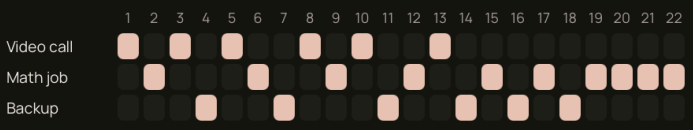
Task	Base	Waiting before	Waiting after	Done before	Done after
Video call	5	0	5	Tick 8	Tick 13
Math job	3	6	12	Tick 16	Tick 22
Backup	1	16	12	Tick 22	Tick 18

Waiting is the number of ticks a task was ready but another task had the CPU. Lower is better for tasks that would otherwise wait a long time.

Base priority only (before)



Dynamic priority (after)



Name this run | Save run

7.5 Demonstration of OS Concepts

OS concept	Where it appears in the project
Process states	Tasks are ready, running, or blocked for input in each tick
CPU scheduling	The runner is chosen at every tick from the ready tasks
Priority	Base priority and the dynamic score in the simulator
Starvation	Backup waits 16 ticks under base priority; the timeline shows the long gap

OS concept	Where it appears in the project
Aging	Waiting tasks gain aging points until they run
I/O behavior	Video call pauses for two ticks after two runs
Fairness	CPU penalty reduces the priority of CPU-heavy tasks
Synchronization (modeled)	The ready list is updated only by the scheduler loop

Chapter -8 Conclusion

8.1 Summary of the Project

This project implemented a Smart CPU Scheduler whose priorities change during execution. The scheduler combines a base priority with an aging bonus, an I/O boost, and a CPU penalty. A simulator compares this policy with static priority scheduling on the same workload. The results are presented as waiting-time tables and timeline charts. The simulator is wrapped in a web application with user accounts, a protected dashboard, saved runs, and a page of course outcomes.

8.2 Objectives Achieved

Objective	Status	Comment
Study scheduling, starvation, and synchronization	Achieved	Covered in Chapter 2
Design a dynamically adjusting priority policy	Achieved	Aging, I/O boost, and CPU penalty
Run both policies on the same workload	Achieved	Side-by-side results in the dashboard
Present results clearly	Achieved	Summary tables and Gantt-style timelines
Build a secure web application	Achieved	Hashed passwords, tokens, per-user data
Implement in C with real threads and system calls	Not achieved	Planned as future work
Measure real processes on Linux	Not achieved	Planned as future work

8.3 Results Achieved

- The starved task finished 4 ticks earlier (tick 18 instead of tick 22) and waited 4 ticks less.
- The interactive task waited 5 ticks more, which shows that the adjustment shifts time from interactive tasks to starved ones in this workload.
- Every user action from sign up to saving and deleting a run was tested through the API and the pages.

8.4 Skills and Learning Outcomes

- Understanding of process scheduling, priority, starvation, aging, and the cost of fairness.
- Experience building a full-stack application with a REST API, a database, and token-based login.
- Practice with secure password storage and per-user access control.
- Experience writing documentation and presenting technical results.

8.5 Limitations and Future Work

The current version has several limitations that suggest directions for future work:

- The scheduler is simulated, so the results describe the model and not a real operating system.
- The simulation handles one CPU and does not model multi-core processors.
- The interactive task is treated only as a task that pauses for input. Real input timing is not modeled.
- The planned C implementation would use fork, pthread, and mutexes on Linux, and would measure real waiting times with a real workload.
- The parameters, such as the aging limit and the boost value, could be tuned automatically to balance interactive and CPU-bound tasks.

Chapter -9 References

9.1 Books

1. Silberschatz, A., Galvin, P. B., and Gagne, G. Operating System Concepts. Wiley. Chapters on CPU scheduling and synchronization.
2. Tanenbaum, A. S., and Bos, H. Modern Operating Systems. Pearson. Chapters on processes, scheduling, and threads.

9.2 Research Papers

No research papers were cited directly in this version of the project.

9.3 Linux / GNU Documentation

1. Linux man-pages project. Manual pages for fork(2), pthread_create(3), and pthread_mutex_lock(3). https://man7.org/linux/man-pages/
2. GNU Project. GCC documentation. https://gcc.gnu.org/onlinedocs/

9.4 Online Resources

1. Node.js Foundation. Node.js documentation, including the SQLite module. https://nodejs.org/api/sqlite.html
2. Express.js. Express documentation. https://expressjs.com/
3. SQLite Consortium. SQLite documentation. https://sqlite.org/docs.html
4. Auth0. jsonwebtoken library documentation. https://github.com/auth0/node-jsonwebtoken
5. Mozilla Developer Network. Web APIs documentation, including Fetch and localStorage. https://developer.mozilla.org/